

PARALLEL COMPUTING — CLASS 0 DIAGNOSTIC

Prerequisite & problem-solving diagnostic • 2 hours

Purpose: This session checks your current programming, algorithmic, data, and systems knowledge. You are not expected to know parallel computing yet. Work individually and show your reasoning.

Instructions: Use a language you know. Do not worry about perfect syntax unless asked. For algorithm questions, explain your thinking. If you do not know something, say so and move on.

Task 1 — Trace the Code (5 min)

Determine the final output. Show the value of x and y after each statement.

```
x = 5
```

```
y = 2
```

```
x = x + y
```

```
y = x * 2
```

```
x = y - x
```

```
print(x, y)
```

```
____x=7    y=14_
```

Task 2 — Find the Bug (5 min)

What does this program actually calculate? What was it probably intended to calculate? Fix it.

correct version _____total+=numbers[i] instead of total=numbers[i]

_____loops overwrites instead of adding it

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
```

```
    total = numbers[i]
```

```
print(total)
```

Task 3 — Nested Loops (5 min)

For the program below: (a) how many times does print() execute? (b) what if both loops run from 0 to 999? (c) what if the outer loop runs 10 times and the inner loop runs 1,000,000 times?

```
10 × 10 = 100
```

```
1000 × 1000 = 1,000,000
```

```
10×1,000,000=10,000,000
```

```
for i in range(10):
    for j in range(10):
        print(i, j)
```

Part B — Algorithms & Complexity

Task 4 — Which Is Faster? (5 min)

Algorithm A scans an array once. Algorithm B compares every element with every element. For an array of size n , estimate the amount of work for each. Which grows faster? What happens when $n = 1,000,000$?

A n operations, B $n \times n$. B is faster. At $n=1,000,000$ that's a million operations for A but a trillion for B — A finishes instantly and B would take many minutes, so B stops at that size

Task 5 — Complexity Ranking (7 min)

Rank these from fastest-growing to slowest-growing as n increases:

n^2 $\log_2 n$ n $n \log_2 n$ 1 2^n

Which become particularly problematic when n becomes much larger?

Task 6 — Searching (7 min)

Given [3, 8, 12, 17, 24, 31, 45, 51, 63], find 45 using two approaches: (1) if you do not know the list is sorted; (2) if you know it is sorted. Explain why the second can be faster.

If I don't know it's sorted I have to check each one in order: 3, 8, 12, 17, 24, 31, 45 — that's 7 checks. If I know it's sorted I use binary search. Middle is 24, 45 is bigger so I throw away the left half. Next middle is 45, found in 2 checks. It's faster because being sorted is extra information — every comparison removes half the list instead of just one item.

Part C — Data & Problem Solving

Task 7 — Large Dataset (8 min)

You have 500 million integers and need to find the largest number. Design an algorithm. How many times does each number need to be examined? Do you need to store all 500 million numbers? Could you process the data in chunks?

Keep one variable for the biggest so far, go through the numbers once, and replace it whenever I see something larger. Each number is looked at exactly once. I don't need to store all 500 million — just the current maximum, so memory stays tiny. Storing them all would need around 2 GB. Chunks work fine because the max of the whole thing is just the max of each chunk's max, so chunks can even be done in parallel.

Task 8 — Counting (5 min)

Given A B A C B A D C A B, design an algorithm producing A→4, B→3, C→2, D→1. What data structure would you use?

Task 9 — Matrix Operations (5 min)

A = [[1,2],[3,4]] and B = [[5,6],[7,8]]. Calculate A+B and A×B. How many individual multiplication operations are required for two 2×2 matrices?

Part D — Systems Intuition

Task 10 — CPU vs Memory (6 min)

A program scans a huge array, but the CPU is extremely fast and the program is still slow. Give three possible reasons.

```
for x in huge_array:
    total += x
```

Task 11 — What Happens When You Run a Program? (7 min)

Describe what happens between typing ./program and seeing the program's output. Use ordinary language. Include anything you think is relevant.

Task 12 — Four Programs (7 min)

Four programs run simultaneously: A uses lots of CPU; B uses lots of memory; C reads a large file; D mostly waits for user input. Identify the likely bottleneck for each. Can all four run at the same time? What determines CPU time? What changes if there is one CPU core?

Part E — Reasoning Challenges

Task 13 — The Slow Program (6 min)

A program processes 1 billion records in 10 hours. A student says: “If we buy a computer that is twice as fast, the program will take 5 hours.” Do you agree? Explain.

Task 14 — Make It Faster (6 min)

A program takes 60 seconds. You may change anything except the answer it produces. List as many ways as possible to reduce execution time.

Task 15 — Bottleneck Identification (6 min)

Execution times are: A=2s, B=2s, C=50s, D=3s, E=2s. Total=59s. If A, B, D and E become 10× faster, how much faster does the entire program become? Where should optimization effort go?

Final Challenge — Design the Algorithm

Task 16 — 10 Million Student Marks (10 min)

You receive 10 million student marks, each between 0 and 100. You must produce: minimum, maximum, average, median, number scoring ≥ 90 , and number scoring < 40 . Design an algorithm; identify data structures; estimate how many times the data must be examined; estimate memory requirements; identify the likely bottleneck; and explain how you would make it faster without changing the answer.

STOP HERE. Check that you attempted every task. You are not expected to know parallel computing yet.